

Graphs in Java

Mostafa Faik

October 27, 2024

Introduction

In this assignment, we explored the `graph` data structure using the Swedish railroad network. A `graph` is a collection of `nodes` (or `vertices`) connected through `edges`. These `edges` form paths between `nodes`, and `edges` can be `unidirectional` or `bidirectional`. In our case, the railroad network is represented as a `graph`, where `cities` are `nodes` and `train routes` are `edges`. This assignment aimed to use `graphs` to determine the `shortest path` between `cities` and explore different techniques to optimize our approach.

We implemented multiple solutions using `Depth-First Search` (DFS), introduced `loop detection` to avoid inefficiencies, and optimized our program with `hashing`. Along the way, we also explored the limitations of the DFS approach and considered the implications for `larger datasets`.

Creating the Graph from CSV

The first task was to read the CSV file *trains.csv*, which lists connections in the form `from,to,minutes`. For this, we needed to represent `cities` and their connections as a `graph`. Each `city` is represented by a `class City`, and each connection is represented by the `class Connection`.

The `city` class stores the name of the city and its connections to neighboring cities. The `connection` class stores the `destination city` and the `travel time` between them.

We implemented the `Map` class to read the CSV file, extract the `city names` and `connections`, and build the `graph`. The *lookup()* method is used to find or create `city` objects as the CSV is processed. The following is an example of the output that got when running the code:

```
Skövde connections:  
-> Herrljunga (35 mins)
```

```
-> Falköping (16 mins)
-> Hallsberg (42 mins)
```

The `connect()` method allowed cities to be connected bidirectionally, meaning that a connection from Skövde to Falköping (16 minutes) also meant there was a connection from Falköping to Skövde.

By breaking the problem into two classes (City and Connection), we made the code easier to manage. Each city can manage its own connections, and we can easily add or remove connections without affecting other parts of the program. Once a city object is created, it knows all its neighbors and the corresponding travel times, allowing quick access to neighboring cities for traversal.

Hashing to Optimize City Lookups

To optimize the process of looking up cities by name, we implemented a hash function using the modulus operator. The goal was to distribute cities evenly in an array to avoid too many collisions and reduce the time complexity of lookups.

```
private static Integer hash(String name, int mod) {
    int hash = 0;
    for (int i = 0; i < name.length(); i++) {
        hash = (hash * 31 + name.charAt(i)) % mod;
    }
    return hash;
}
```

The mod value was chosen based on the number of cities and our desire to spread them evenly across the hash table. We chose various values for mod and analyzed the distribution of cities. Using a mod value of 100, 60, and 70, we noticed some collisions but overall efficient lookups. The following is an example with $mod = 60$:

```
0: Kalmar Ludvika Helsingborg
3: Fagersta
4: Örebro Storvik
...
```

In this output, cities like Kalmar, Ludvika, and Helsingborg share the same bucket, which is an example of a hash collision.

Pros of Using Hashing:

- Fast lookups: Using a hash function allows us to access cities in constant time on average, which is essential for larger datasets.
- Reduced search complexity: Without hashing, we would have had to search through a list of cities, leading to an $O(n)$ complexity. Hashing reduces this to $O(1)$ for most cases.

Cons of Hashing:

- Collisions: If two city names hash to the same value, we experience a collision. We need to handle collisions either by using buckets or by probing for the next empty slot in the hash table.
- Memory trade-off: Depending on the hash table size, we may have wasted space (if the table is too large) or experience performance degradation (if the table is too small and causes many collisions).

Shortest Path from A to B (Naive Approach)

The core problem in this assignment was finding the shortest path between two cities. We started with a naive depth-first search (DFS) approach. This method recursively explores all possible paths from the starting city to the destination, keeping track of the shortest one found within a specified maximum time (to avoid infinite loops).

We tested the program by calculating the shortest paths for various routes such as Malmö to Göteborg, Göteborg to Stockholm, and others.

Shortest path from Malmö to Göteborg is 153 min (442 ms)

Shortest path from Göteborg to Stockholm is 211 min (193 ms)

Shortest path from Malmö to Stockholm is 273 min (213 ms)

This output shows that the algorithm performs well for shorter routes but struggles for longer paths. For example, it fails to find a valid route from Stockholm to Umeå.

Example Output Explanation (DFS):

- Malmö to Göteborg (153 min): DFS quickly finds the shortest path as these two cities are connected by a series of short connections.
- Stockholm to Umeå (No Path): DFS struggles with long-distance paths, especially when circular paths and dead ends are present. It fails to find the path within the allowed time, highlighting the inefficiency of DFS for certain routes.

Detecting and Avoiding Loops

One problem with DFS is that it can revisit `cities` multiple times, potentially leading to `infinite loops` or redundant calculations. To avoid this, we modified our `algorithm` to detect `loops` by tracking the `cities` visited along the current `path`.

We added an array `path[]` to store the visited `cities` and checked for each `city` if it had already been visited.

```
for (int i = 0; i < sp; i++) {  
    if (path[i] == from)  
        return null; // Loop detected  
}  
path[sp++] = from;
```

The `algorithm` avoids re-exploring `cities` it has already visited, reducing unnecessary computation. The solution is straightforward, before visiting a `city`, we simply check whether it's already in the `path`.

If DFS attempts to revisit Malmö after traveling to Lund, the loop detection mechanism halts the `search`, preventing it from exploring the same `cities` repeatedly. This improved the performance of the `algorithm` by avoiding unnecessary revisits, although it didn't entirely solve the inefficiency for `larger graphs`.

With loop detection, we reduced the `time complexity`, and the search results were more efficient. However, the problem persisted for longer `routes` such as Umeå to Göteborg, where no valid `path` was found within the `time limit`.

Performance Benchmarking

We benchmarked the DFS-based `search method` to assess its performance in finding the `shortest paths` between selected `city pairs`. The table below lists the `shortest path distances` and the `time taken` for each `search`. Generally, the `algorithm` performed well for `shorter paths` but exhibited limitations for `longer distances` due to the `depth-first` nature, where `longer paths` took substantially more time to complete.

From	To	Shortest Path (min)	Time (ms)
Malmö	Göteborg	153	442
Göteborg	Stockholm	211	193
Malmö	Stockholm	273	213
Stockholm	Sundsvall	327	141
Sundsvall	Umeå	190	551
Göteborg	Umeå	705	480
Umeå	Göteborg	705	264

Table 1: Benchmark results for Naive approach

We can observe from the table that:

- Performance on Shorter Paths: The **algorithm** is efficient for **shorter trips**. For example, the **path** from Malmö to Göteborg took *442 ms*, and Göteborg to Stockholm took only *193 ms*. These results indicate that DFS can work well for **shorter routes** or regions where the **destination** is relatively close to the starting point.
- Increasing Time with Path Length: As expected, **paths** covering **longer distances** required more time. For example, the **path** from Sundsvall to Umeå took *551 ms*, while Göteborg to Umeå took *480 ms*. The **search** time grows with the **path length** due to the increasing number of recursive calls and backtracking steps.
- DFS Limitations in Complex Graphs: The **algorithm's** performance degrades significantly when exploring larger sections of the network, as demonstrated in the **path** from Umeå to Göteborg. This route took *480 ms* to compute, showing DFS's inherent inefficiency when there are many **nodes** and longer **paths** to search.

In DFS, each **node** may need to be visited, making the worst-case **time complexity** approximately $O(V + E)$ for **vertices** V and **edges** E . For a **cyclic graph**, however, this **complexity** rises since DFS may explore multiple **routes** back to the same **node** unless managed by **loop detection**.

Overall, these **benchmarks** highlight DFS's limitations in handling **longer routes** within complex **graph structures**. The **depth-first search** approach lacks an effective way to remember **paths** it has previously evaluated, leading to increased search times as the number of **nodes** increases.

Things to Ponder

While our solution works for the current **graph** of *52 cities*, it would become increasingly inefficient as the number of **cities** increases. The

primary problem is that our algorithm does not remember paths it has already calculated. For example, if the algorithm finds the shortest path from Stockholm to Kiruna, it should store this result and reuse it when calculating paths from other cities that pass through Stockholm.

If we expanded the map to a European scale, our current depth-first approach would be too slow to find the shortest paths efficiently. The algorithm would need to explore too many possibilities, most of which wouldn't lead to the shortest path. A more advanced algorithm like Dijkstra's would be needed to handle larger graphs.

Conclusion

In this assignment, we explored the complexities of graph traversal by finding the shortest paths in Sweden's train network. We started with a naive depth-first search approach, which was functional but inefficient for longer routes. By introducing loop detection, we improved performance but still encountered limitations for larger paths. Ultimately, we discovered that while DFS can work for smaller graphs, it's not the best choice for finding shortest paths in large networks due to its inability to remember previously discovered paths.